计算机学习地图 · 实习冲刺版

把所学变成面试证据

Java 后端为主线，AI 应用为加分项：
8 / 16 / 24 周可执行手册，目标是通过初筛与技术面。

⏱ 8–24 周，按你的起点选　🎯 目标：简历过筛 + 技术面　📦 每周交三样：代码 · 验证记录 · 讲解

📅 更新于 2026-10-19

> 面试官不关心你用了多少技术，只关心出事时你知不知道往哪看。
> 这份手册只做一件事：把「我学过」变成「我能证明」。

长线版 · 20–26 个月　实习冲刺版 · 8–24 周

1. 今天开始，用交付物换掉收藏夹

这节帮你拿回准备方向分：每天结束时，要有代码、测试或一次脱稿讲解留下来。

“学会”是关教程能改需求，关 AI 能解释，弄坏还能定位。目标是争取初筛和技术面试机会，不承诺录用。

主线 16 周，零基础选 24 周，有 Java、SQL 和接口经验才选 8 周。阅读约 25–30 分钟，周表随用随查。

⚠ **注意**

周数是训练预算。每周只有 8 小时就拉长日历，别用看完视频冒充完成。先核对招聘窗口、学历和到岗要求，学习不能替代资格。

暂缓组成原理、CSAPP、MIT 6.1810、CS144；删去 JVM 调参、字节码和类加载全流程；不学一致性算法、DDD、微服务全家桶、K8s、消息队列集群。先交付可靠服务。

2. 面试官在寻找哪些证据

这节帮你把“我学过”改成“我能证明”，避免会背概念却接不住追问。

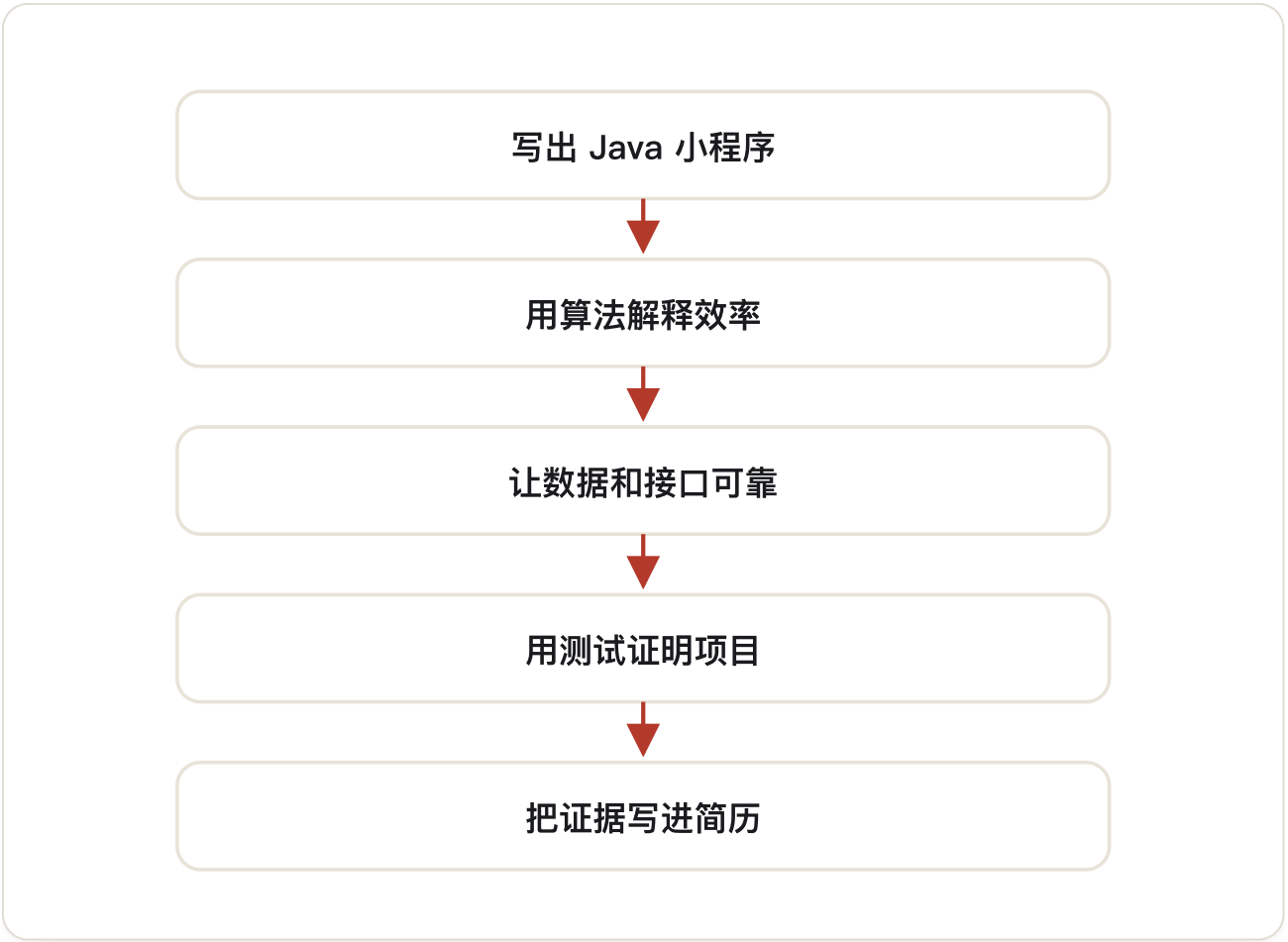
以下是本文评审判断，不是公司打分表；题目是训练问法，不是真实面经原题。

判断	面试里怎样留下证据
能否独立完成小需求	当场给接口加一个边界校验，解释失败时返回什么。
是否理解因果	讲缓存时说出数据库更新后为何可能读到旧值，再给取舍。
是否拥有项目	指出自己修过的一个 bug，展示复现条件和回归测试。
能否合作、承担责任	不会时说明已知边界和验证步骤，不把 AI 或队友当挡箭牌。

面试间里的一幕：候选人把 HashMap 的扩容讲得很熟，我随口问“你往里面放自定义对象，取出来却是 null，为什么”。他卡住了三秒，说“可能是版本问题”。

不是他不会——是他背的是**结论**，没碰过**因果**。我当场让他写 `equals` 和 `hashCode`，写完他自己笑了：“原来必须一起重写。”

这也是本文的取舍标准：**凡是只能背、不能当场验证的，优先级都往后放。**

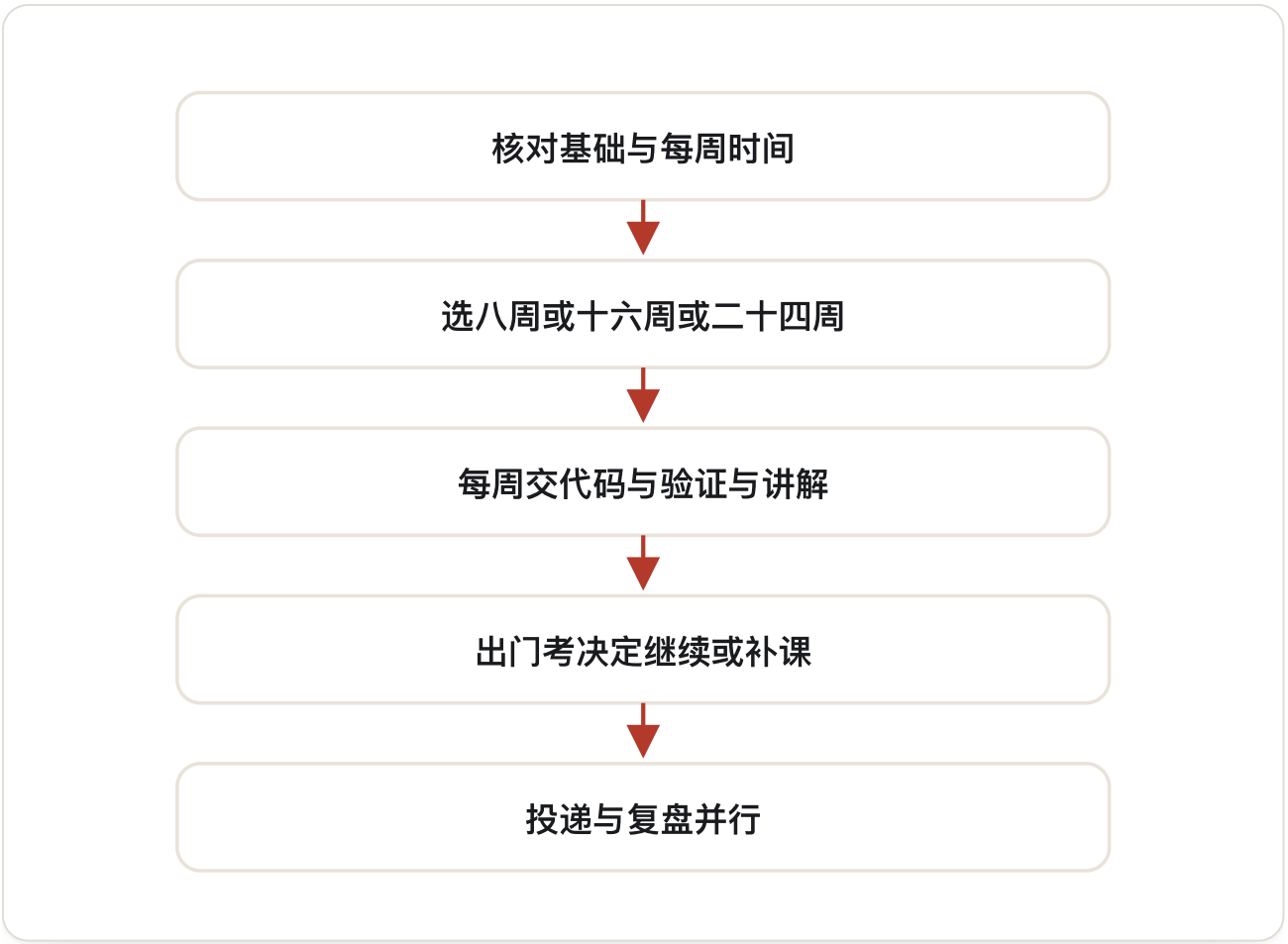

写出 Java 小程序
用算法解释效率
让数据和接口可靠
用测试证明项目
把证据写进简历

3. 选一条时间线，每周交三样东西

这节帮你拿执行分：周末用可运行成果验收，不用学习时长安慰自己。

每周交代码、验证记录、录音。题量是新题预算，另复做错题，记录边界、复杂度与错误原因。卡住 20 分钟可看提示，隔天关答案重写。

没完成交付就顺延，先砍美化、多框架和 Agent 扩展。项目可复现、简历不虚构后，就小批投递，按反馈调整。



8 周版：已有基础，每周约 25–30 小时

这档帮你补证据缺口；写不出循环统计、SQL 查询和接口，就换较长档。

周	本周必须交付
1	写文件词频统计，补空文件测试；做 8 道数组 / 哈希题；录 3 分钟集合选型。
2	写有界线程池实验、复现计数竞争；做 8 道链表 / 栈题；解释一次拒绝任务。
3	给资料表造数据、比较索引前后计划，复现事务回滚；做 8 道二分题；讲扫描差异。
4	交付带登录、权限、分页的资料 API，复现并修复事务失效；做 8 道树题；画请求链路。
5	加缓存并测试过期、停机、重复提交；做 6 道堆 / 窗口题；录 5 分钟项目介绍。
6	AI 方向做带引用的检索问答和 20 条评测；后端方向做等量接口故障用例；做 6 道搜索题。
7	从空环境复现项目，写一页简历，小批投递；做 6 道动态规划题；模拟面试一次。
8	按反馈修两项最大短板；限时重做 8 道错题；再模拟两次并更新投递记录。

16 周版：会基本语法，每周约 15–20 小时

这档帮你串联知识，第 7 周起迭代主项目。

周	本周必须交付
1	配好 JDK、Git、Maven；写命令行任务清单和 3 个测试；录一次断点定位。
2	加文件保存、接口和异常处理；做 5 道数组题；解释坏输入怎么处理。
3	用集合做词频统计，画 HashMap 冲突图；做 6 道哈希题；演示错误键设计。
4	写限额线程池实验，复现并修复计数竞争；做 6 道链表 / 栈题；讲锁与可见性。
5	写建表及 8 条查询，比较联合索引；做 5 道二分题；留存执行计划。
6	双连接观察隔离、回滚和锁等待；写缓存失效实验；做 5 道窗口题并讲一致性。
7	建资料管理 API，带参数校验、分页和错误码；做 5 道树题；画 HTTP 请求路径。
8	加登录与所有者权限，复现并修复自调用事务问题；做 5 道树题；交越权测试。
9	加缓存，覆盖穿透、击穿、雪崩的缩小实验；做 5 道堆题；说明降级边界。
10	加重复提交保护，压测固定读接口；做 5 道搜索题；保留吞吐与延迟原始记录。
11	AI 方向做文档切分与检索基线；后端方向修一条慢查询；做 5 道回溯题并讲取舍。
12	AI 方向加引用与拒答、完成 20 条评测；后端方向完成 20 条接口边界用例；做 5 道动态规划题。
13	复现超时、数据库断连或模型失败，补日志和回归；重做 6 道错题；写故障记录。
14	从空环境复现，录 5 分钟项目介绍；重做 6 道错题；写一页简历并开始投递。
15	做两次限时面试，修两项共性失分；重做 6 道错题；按 JD 调整简历证据顺序。
16	再做两次完整模拟，核对简历每句话；重做 6 道错题；依据回复继续投递。

24 周版：零基础起步，每周约 12–16 小时

这档帮你打基础；第 8 周仍写不出程序，就延长基础段。

周	本周必须交付
1	安装 JDK 和编辑器，运行一个问候程序；改坏再修好；讲清源码如何运行。
2	写条件、循环和数组练习共 6 个；手算边界输入；用断点跟踪一次循环。
3	写命令行记账器，拆成 3 个方法；加非法金额校验；录 2 分钟数据流。
4	把记录改成对象，为保存动作定义接口；替换一种实现；解释对象与接口。
5	加列表、去重和分类统计；做 4 道数组题；解释 List、Set、Map 的选择。
6	加文件保存和异常处理；写 4 个测试；演示坏文件不让程序直接崩掉。
7	用 Git 分支修改统计规则并检查差异；做 4 道哈希题；写复现说明。
8	关教程重写小工具核心，给陌生人按说明运行；复做 4 道错题；录 3 分钟讲解。
9	做词频统计和 HashMap 冲突实验；做 5 道链表题；画查找过程。
10	做计数竞争与有界线程池实验；做 5 道栈题；录锁与 volatile 的差别。
11	建资料表并写 8 条 SQL；做 5 道二分题；交索引前后执行计划。
12	用两个连接做回滚、隔离和锁等待实验；做 5 道窗口题；解释事务边界。
13	写资料查询 API 和分页校验；做 5 道树题；记录请求、响应和状态码。
14	加登录、所有者检查与事务；做 5 道树题；交越权和回滚测试。
15	加缓存并复现热点失效；做 5 道堆题；说明旧值与数据库压力的取舍。
16	加重复提交保护与超时处理；做 5 道搜索题；写一次失败请求的定位记录。
17	固定数据量压测一条接口并改一个瓶颈；做 5 道回溯题；保留前后原始数据。
18	AI 方向做检索基线；后端方向做故障恢复实验；做 5 道动态规划题并讲替代方案。
19	AI 方向加引用、拒答；后端方向补接口边界；各交 20 条用例及失败分类。

周	本周必须交付
20	修两类失败，保留一组未参与修改的验收题；重做 6 道错题；讲结果的局限。
21	从空环境跑项目，补操作说明和演示；重做 6 道错题；录 5 分钟介绍。
22	写一页简历，小批投递；模拟一次并修一个失分点；重做 6 道错题。
23	模拟两次，给项目连续追问 10 轮；重做 6 道错题；更新简历和投递记录。
24	对照 JD 补最大缺口，再模拟一次；重做 6 道错题；定下后续两周行动。

零基础（24 周档）：每周先做到这个最低标准，再加量

上面每一行的第一件事，都可能让你卡住。先照下面这个「最小可运行」标准做出来，再谈优化：

第几周	最小骨架（照这个先跑通）
1	终端里 `javac Hello.java && java Hello` 能出结果；再写一个 `for` 打印 1–10
2	`List<String> names = new ArrayList<>();` 增删查一遍，用 `System.out` 验证
6	一张 `resource` 表 + 一条 SELECT；能说出这行 SQL 走了哪个索引
12	接口能返回 JSON：`curl localhost:8080/api/resources` 拿到 200
18	空数据库、按 README 步骤，10 分钟内把项目跑起来
21	五分钟讲完项目，录音回放，能听出自己在哪一段心虚

卡住超过 20 分钟就去看提示，**第二天关掉答案重写一遍**——这是本文唯一要求你反复做的事。

4. 六个阶段，过出门考再往下走

这节帮你拿基础分：用实验和项目回答问题。

P0 写得出，也调得动

这阶段帮你拿编码起步分。**为什么学**：先能改小程序，才不至于照抄框架代码。

学什么

变量像贴标签的盒子，是保存值的名字。先用循环打印 1 到 10，再改条件，亲眼看输出怎么变。

对象像带操作按钮的资料卡，是数据与行为的组合。`List<Student>` 里塞自定义类，就是最常见的用法。

接口像插座规格，是调用约定。换实现不用改调用者——这句话在面试里能顶半页纸。

异常像报警，是失败报告机制。写之前先想清楚：哪些输入该拒绝，哪些故障该上抛。

Git 像账本，管版本；**Maven** 像装配清单，管依赖与构建；**测试**像验收尺，断言结果符合约定。这三样不用精通，但要跑通一次完整循环。

面试会怎么问：① 给记录加一个字段，要改哪里？② 文件不存在怎么处理？③ 接口与实现拆开有什么收益？④ 如何定位这段空指针？

答不上来会怎样：现场修改卡住，项目代码难以自证。

出门考：不看教程写“文件读入→分类统计→输出”；覆盖空文件、坏数据、重复记录；断点定位一个故意制造的 bug，并讲 3 分钟。

资源：dev.java 读语法、对象、异常。

P1 集合、算法与并发的为什么

面试官问「你 HashMap 里放了自定义对象，为什么取不出来」——这一节就是为这一问准备的。为什么学：效率和线程安全会直接影响项目正确性。

学什么

集合像不同的收纳盒：List 保顺序、Set 去重、Map 按键找值。面试时**先讲选型理由，再讲代价**，别背接口签名。

HashMap 像按门牌分区的快递柜，是用散列定位桶的映射。标准答法：先算桶位置，再比较键；冲突要继续找，扩容有成本。

`equals` 与 `hashCode` 是一对——`equals` 相等的键，`hashCode` 必须相同。键放进去以后再改参与计算的字段，**可能再也找不到它**（上面那幕就是这件事）。别只记"要一起重写"，要能说出为什么。

复杂度像估算"人数翻倍后排队要多久"，是规模与计算量的增长关系。先给朴素解，再说怎么减少重复工作；写完补上空输入、重复值、边界三组测试。

线程像同时办事的工人，共享内存但各有执行路径。**锁**像厕所门栓，保证临界区互斥；**volatile** 像及时更新的公告栏，管可见性和顺序，但**不能把 `count++` 变原子**——这句话答错，后面全垮。

线程池像有候客区的窗口，复用线程、排队管理任务。沿着"核心线程 → 队列 → 最大线程 → 拒绝策略"回答；无界队列可能把内存吃光。容量按耗时、阻塞比例和实测来定，**不是线程越多越快**。

面试会怎么问：① HashMap 冲突后怎么找？② 复合操作用了并发 Map 就安全吗？③ volatile 为什么防不住丢计数？④ 队列满了会怎样？⑤ 这题能少遍历一次吗？

答不上来会怎样：缓存键和并发请求讲不清；手撕换边界就出错。

出门考：画一次冲突与扩容；复现计数竞争并用锁修好；制造线程池拒绝；30 分钟独立写一道练过的中等题、解释复杂度并补边界测试。

资源：线程池 API 查排队；Hello 算法 按周表选题。

P2 MySQL 和 Redis：数据不能靠祈祷

「这条查询为什么慢？」——学完这节，你能在面试现场打开 EXPLAIN 当场回答。为什么学：能返回不代表查得快、写得对。

学什么

索引像书的目录，用来加速定位。回答时讲**联合索引的列顺序为什么这么排**，再用 EXPLAIN 把扫描行数摆出来——别说"加了索引就快"，那是外行话。

B+ 树像多层目录，叶子节点相连，所以范围查询也快；代价是空间和写入维护。

覆盖索引像目录里已经把答案写全了，不用再翻正文；**回表**就是拿着主键回去取整条记录。面试官爱听这个对比："只查索引内的列"和"多查一个非索引列"分别多少行。

事务像一次成套转账，要么一起提交、要么一起回滚。**隔离级别**规定并发事务能看见什么。**MVCC** 像文档的历史版本，按可见性规则给你一个"快照"。用两个连接做实验，分清快照读和锁定读——别背"RR 永不幻读"，那句话经不起追问。

缓存像桌边便签，是数据库之外的快速副本。穿透是查不存在的数据，可用短期空值缓存并限制滥用；击穿是热点失效，可合并并发回源；雪崩是大量缓存同时失效或服务故障，要错开过期并限流降级。

数据库写完再删缓存仍有竞争窗口，不能承诺强一致。回答要说允许多旧、过期如何兜底；订单状态等关键判断回数据库核实。Redis 停机时全量回源也可能压垮数据库。

面试会怎么问：① 联合索引为什么这样排列？② 执行计划怎么看？③ 转账中途失败会怎样？④ 缓存三种故障有什么不同？⑤ 数据更新后仍读到旧值怎么办？

答不上来会怎样：优化无依据，重复写入和脏数据兜不住。

出门考：造足以比较差异的数据，保留索引前后计划；双连接复现回滚与锁等待；模拟热点失效和 Redis 不可用，验证接口有超时与降级。

资源：MySQL 索引手册 对照自己的 SQL；Redis 键与过期 用来做失效实验。

P3 HTTP、Spring 与排错

项目深挖十有八九从「这个接口怎么工作」开始，这节让你能一路讲到数据库。为什么学：项目深挖通常从"这个接口怎么工作"展开。

学什么

HTTP 像业务信件，是应用层的请求/响应协议；**TCP** 像带编号和确认的运输通道，保证可靠有序的字节流。握手负责建连和确认初始序号，挥手关闭两个方向——报文可能合并，这也是为什么抓包时看到的时序和你以为的不一样。

字节流没有消息边界，得靠长度或分隔符切分。HTTP 本身无状态，登录靠 Cookie 带会话标识；HTTPS 在中间加一层 TLS 保护传输。

认证和授权要分清楚：认证是“你是谁”，授权是“你能看谁的资料”。顺带答一句“超时重试为什么可能重复写”——很多人只会说“要幂等”，说不出为什么。

IoC 像食堂统一配餐：容器创建、管理对象并注入依赖。**AOP** 像门口统一查票：在业务调用周围织入通用逻辑。回答时画一条 `Controller → Service → 数据库`，把校验、事务、异常处理标在正确的位置上。

事务代理像必须经过的验票口。默认代理模式下：同类内部自调用会**绕过代理**；吞掉异常可能让代理无法按预期回滚；默认回滚规则也不覆盖所有受检异常。结论：**“加了注解” ≠ “事务生效”**。

JVM 像车间：堆里放对象，栈里存调用帧，GC 回收不可达对象。练的是排查路径——CPU 高查忙线程，内存涨查对象保留，响应慢查依赖。**先留证据，再改参数。**

面试会怎么问：① 请求超时能直接重试吗？② TCP 为什么要确认序号？③ IoC 与自己 new 有何区别？④ 自调用事务为何没生效？⑤ CPU 高时先看什么？

答不上来会怎样：能背注解却不能定位异常；权限或事务漏洞会削弱整个项目的可信度。

出门考：交付校验、分页、登录和所有者检查；证明用户甲不能读乙的资料；写事务回滚集成测试；用线程转储定位一个自造忙循环。

资源：MDN HTTP 看报文；事务注解说明 核对代理与回滚。

P4 AI 应用：先检索，再证明有效

这阶段帮你避免把“接过模型接口”写成 AI 工程能力。**为什么学**：投 AI 应用岗要证明质量、失败处理和成本控制。

学什么

RAG 像开卷考试：先检索资料，再让模型照着答。面试要能区分两种失败——**“找错了资料”**和**“资料对了但答错”**，因为改法完全不同。

Embedding 像给文本画语义坐标，把内容变成向量。记住一句：**向量相近不等于事实正确**。

Prompt 工程像写任务委托书：讲清输入、约束、示例、输出格式，然后用同一批题迭代。面试时展示“我改了什么、同一批题的结果怎么变”，而不是背万能咒语。

Agent 像会自己决定下一步的助手：模型参与决定调什么工具、走哪条路。但**固定流程通常更容易测**——所以要能回答“为什么这里真的需要自主选择”，以及步数上限、超时和失败出口在哪。

MCP 像统一插口，是连接 AI 应用与工具、资源的协议。它**不自动给权限**，也不是 Agent 的别名。面试要说明：工具参数校验、访问控制、敏感操作二次确认，分别在哪一层做。

评测像固定试卷，用同一把尺子比较系统。标 20 条：正常 / 无答案 / 歧义 / 越权 / 诱导指令，各 4 条；记录资料命中、引用支持、是否拒答、耗时。**另外留一批没见过的题做验收**——只优化见过的题，等于背答案。

面试会怎么问：① 没检索到和模型胡答怎么区分？② 分块变大有什么代价？③ 引用存在就算答对吗？④ 为什么不用固定流程？⑤ 恶意文档让工具导出数据怎么办？

答不上来会怎样：项目像接口包装；“准确率高”没有口径就不构成证据。

出门考：同一份非敏感资料跑关键词检索基线与一次改进；保留逐题结果及失败原因；展示无依据拒答和越权阻断。没实调模型时标“模拟响应验证”，不能把它当效果评测。

资源：Spring AI 学接入；MCP 区分各角色。框架只学一个。

P5 项目交付与面试表达

面试官不会替你从仓库里找贡献——这节教你把贡献摆到台面上。为什么学：面试官不会替你从仓库里找贡献。

学什么：复现说明像菜谱，让别人得到同一结果；回归测试像修桥后复检，检查修复是否破坏原行为。回答要串起需求、实现、测试与证据。

面试会怎么问：① 哪部分由你负责？② 为什么没选另一种方案？③ 指标如何测？④ 今天需求变了你改哪？⑤ 让别人运行需要几步？

答不上来会怎样：技术栈越多，无法自证的漏洞越多。

出门考：从空环境按说明运行；5 分钟脱稿介绍；接受第 5 节十轮追问；简历每条都能定位到代码、测试或记录。

资源：用 Git Book 整理可读的修改历史。

5. 一个项目，讲五分钟，追十轮

> **项目不是讲出来的，是「跑得起来 + 说得清」证明出来的。**

做“带权限的资料库”：保存、分类、检索资料。后端方向做可靠查询与缓存，AI 方向在同一服务上加带引用问答。

幂等像快递单号，重扫不应重复发货；定义是重复请求不额外改变结果。回答用户范围内的请求标识、唯一约束、重复响应和过期规则，别只放个内存标记。

迭代	完成标准
能用	资料新增、分页、分类可演示；非法输入有稳定错误响应。
可靠	登录、所有者检查、事务和重复提交都有失败测试；日志能关联一次请求。
可交接	有启动步骤、配置占位符、测试命令、限制说明和短演示；没有密钥与个人资料。

P95 像排序后靠后的等待时间，是 95% 请求不超过的延迟分位数。报告写机器、数据量、并发、预热、时长和错误率；本机实验别写“生产级”。

五分钟练习预算：半分钟讲痛点，一分钟画路径，一分半讲难点，一分钟讲测试与指标，一分钟讲局限和下一步。

十轮追问按因果走：① 谁使用？② 数据怎么存？③ 为什么选这张表？④ 为什么这个索引？⑤ 缓存旧了呢？⑥ 两次提交呢？⑦ 他人越权呢？⑧ 依赖超时呢？⑨ 测试怎么证明？⑩ 不用当前方案还有什么选择？

面试间里的一幕：候选人讲了四分钟项目，从「微服务架构」讲到「智能中台」。我插一句：「这个接口慢的时候，你先看什么？」

他沉默五秒：「看监控。」

我换个问法：「假设没有监控，你手上只有 EXPLAIN 和日志，你干嘛？」——他答上来了，而且答得很好。可惜前四分钟已经浪费了。

结论：面试官不关心你用了多少技术，只关心**出事时你知不知道往哪看**。所以叙述顺序应该是：痛点 → 一次真实故障 → 你怎么定位 → 改了什么 → 指标变化。

反例：“高并发智能中台，性能提升 90%。”却说不出负载与原始数据。改写为“比较资料分页索引前后执行计划，保留复现步骤”，实测后再填结果。

6. 一页简历与 JD：关键词必须带证据

这节帮你减少初筛信息缺失，不能绕过资格要求。

初筛像配零件，判断经历是否符合岗位。学历、专业、毕业时间、实习时长、到岗时间可能是硬条件；关键词帮助检索和阅读，不同团队权重不同，没有保证通过的公式。

一页按“联系与方向→教育与到岗→经历 / 项目→技能证据”排。有相关实习优先展示，无实习就突出项目；成绩、排名只写真实相关的，不伪造学历与工作量。

```
姓名｜求职：Java 后端实习｜联系方式
学校 / 专业 / 毕业时间｜可到岗日期 / 每周天数 / 连续月数
项目：服务谁、解决什么问题（两行以内）
贡献：问题 → 我的动作 → 测试或实测结果 → 证据入口
技能：能用 HashMap 分析查找，能用 EXPLAIN 验证索引等
仓库或演示：可访问入口；保留运行说明，不含密钥
```

反例：“熟悉 Java、MySQL、Redis”。改为“实现用户隔离接口，验证越权、回滚和重复提交”。个人项目别写成企业任职。

JD 像采购清单，是岗位职责与条件说明。本文无全市场统计，不把某词一概当凑数；看实际交付要求，再看“了解 / 优先”。

关键词	怎样判断是否要重点准备
RAG	要做知识库就准备切分、召回、引用与失败分析；只列词时问清任务。
Agent	要做多步任务就准备状态、失败恢复和上限；解释固定流程是否够用。
MCP	要接工具就准备协议角色、权限与参数校验，不堆服务。
评测	要改善效果就拿出逐题结果、标注标准与对照。
Prompt 工程	要稳定输出就准备格式验证、失败样例与回归，不比谁提示长。

投递记录不是给 HR 看的，是**给你自己复盘用的**。没有记录，你的复盘就只是在猜。

日期	公司/岗位	简历版本	结果	下一步
例：10-20	A 厂 · Java 后端实习	v2（突出索引优化）	简历挂	核对 JD 是否要求学历/到岗
例：10-23	B 厂 · AI 应用实习	v3（突出评测）	一面挂 · 问倒 RAG	补「检索失败 vs 生成失败」

投递表记岗位、资格、关键词、简历版本、日期与反馈。没回复先查匹配度；技术面反复失败再分类补课。

7. 面试流程：把时间花在能验证的地方

> **面试只有一小时，这节教你把一小时花在能被验证的地方。**



实际轮数与顺序因岗位而异，八股与手撕无统一比例。60 分钟模拟可分：介绍 3 分钟、项目 20 分钟、基础 12 分钟、编码 20 分钟、反问 5 分钟。这是练习建议，不是统计。

介绍讲身份、可实习多久、做过什么、值得追问哪个点。例：“我的资料库处理了重复提交与缓存失效，有失败用例，可以展开。”没做过就别照念。

八股讲类比、定义和实验；手撕确认输入输出和边界，给朴素解与优化，再编码测试。不确定就提出反例验证，别编结论。

反问：“实习生前四周交付什么？”“谁负责评审和测试？”“更看重检索效果还是后端可靠性？”让回答帮助你选岗位。

8. AI 协作：用过工具，也要拥有代码

这节帮你保住可信度：AI 生成的代码也要能解释。

Vibe coding 像让助手按口述搭积木，用自然语言推动代码生成。原型可以快；进项目要 review、测试、验证依赖。这是分工建议，不是团队制度统计。

面试会检查理解与验证。提交的每一行都应能解释目的、输入输出、失败条件与替代方案；不要求背第三方库源码。

面试官真正怕的不是你用 AI，而是你说不清为什么这么写——因为那意味着线上出事时，你修不了。

你定验收→AI 给小步方案→你看 diff→跑测试→独立改需求。Review 像交稿前审阅，核对变化与需求；测试只覆盖设计过的情况，不失败不等于无缺陷。

> 需求：给我的资料接口增加重复提交保护。
> 先指出现有调用路径与不确定之处，不要直接大改。
> 给两个方案，说明并发、过期和失败重试的差异。
> 我选定方案后，只改必要部分，并列出可复现的失败测试。
> 不要读取或输出密钥，不要添加未核实的依赖。
> 完成后逐段解释；由我独立修改一个边界场景再验收。

问清获准工具、数据能否外发、谁 review、必过测试。内部代码、用户资料与凭据按团队规则处理，提交者仍对缺陷负责。面试用 AI 先获允许。

被问穿时：“初稿由 AI 生成，我验证了这些路径；这里理解不够，我给一个可解释的简化方案。”面后删掉无法自证的表述，补测试、独立重写核心。

9. 80 / 20 清单：投递前的最低证据

> **投递前用这张表查一遍短板——它不保证通过，但能告诉你输在哪。**

验证后才打勾。进度保存在本浏览器，与长线版分开；投递前重新抽测。

P0 独立编码

- [x] ~~关教程完成文件统计，能处理坏输入并断点定位。~~
- [ ] 能用 Git 解释一次差异，别人按说明可运行程序。

P1 基础推理

- [ ] 画 HashMap 查找与冲突，解释集合选型和可变键风险。
- [ ] 复现计数竞争与任务拒绝，说明锁、volatile 和队列边界。
- [x] ~~限时完成数组、链表、树的代表题，并解释复杂度和边界。~~

P2 数据可靠

- [ ] 用执行计划证明一次索引选择，双连接验证事务边界。
- [ ] 演示缓存三类故障的区别，说明一致性与降级限制。

P3 接口可靠

- [ ] 讲清 HTTP / TCP 与请求路径，验证事务失效的修复。
- [ ] 越权、重复提交、超时都有失败测试和可定位日志。

P4 岗位加分

- [ ] AI 方向交引用、拒答与逐题评测；后端方向交故障实验与回归结果。

P5 面试证据

- [ ] 项目能讲 5 分钟并接十轮追问，指标有原始记录。
- [ ] 简历一页且每条可自证，完成至少两次模拟并修复共性问题。

10. 常见死法：失败要能定位

这节帮你止损：把拒绝变成能修的缺口。

失败表现	下次行动
学了一堆却没有能运行的项目	暂停新课，先交一条从请求到数据库的完整路径。
项目讲架构，追问只说“框架帮我做了”	选一条请求画调用路径，逐层做一次失败实验。
刷题只记答案，换输入就错	先手推边界，再关答案重写；记录识别特征和反例。
简历写满 AI 名词，没有效果证据	删除未做内容，补同一题集上的对照与失败样例。
不会却硬猜，越答越矛盾	明确已知与未知，提出最小验证步骤。

11. 版本与来源：让材料可核对

这节帮你避免版本混用；运行依赖以实际项目为准。

2026-10-19 核验：JDK 25 LTS、Spring Boot 4.1.x、Spring AI 2.0.x、LangChain4j 1.22.x。LTS 是长期支持标记，具体条款看发行版。

依据为 Oracle 支持路线、Boot 4.1.1 发布、Spring AI 2.0.1 发布、LangChain4j 1.22.0 发布。Spring AI 官方入门说明支持 Boot 4.0.x / 4.1.x；LangChain4j 部分模块带 beta 后缀，按模块文档选，别统一替换数字。

本文未实跑这些 Java 组合。建项目时锁定补丁版本、跑最小测试；外链本次 curl 均返回 200，未来可能变化。

关于这份文档：主骨架来自飞书《后端学习路线》（花绿青），由本仓库长线版再编。AI 应用开发及求职训练为方法论外推，不是面试题库或录用保证。沿用 CC BY 4.0。